Heuristic A* Search

for Large Sliding Puzzles

Manhattan vs Linear Conflict vs Disjoint Pattern Databases

Anthony Kwasi

CS 57200 - Heuristic Problem Solving | Track B: Search & Optimization

Purdue University - Spring 2026

Agenda

1	**Problem & motivation**	Why the 15-puzzle still matters
2	**Background**	A*, IDA*, admissible heuristics
3	**System design**	Pluggable heuristic architecture
4	**Three heuristics**	Manhattan, Linear Conflict, Disjoint PDB
5	**Experimental setup**	Ablation methodology and seeds
6	**Headline result**	15.2x speedup at depth 50
7	**Analysis & limitations**	Why PDB wins; what is left
8	**Future work & demo**	Where to go next

The 15-puzzle: a canonical hard search problem

~10^{13}

reachable states on 4x4

~80

maximum optimal depth

~2.13

average branching factor

Why it matters

- Brute-force BFS exhausts memory on 4x4.
- Optimality requires admissible heuristics.
- Direct analogue for warehouse routing, robot path planning, and game AI.
- Benchmark for evaluating heuristic quality on the same instances.

Background: A*, IDA*, and admissibility

A* search

Best-first with f(n)=g(n)+h(n). Admissible h yields optimal solutions; consistent h avoids re-expansion.

Hart, Nilsson & Raphael (1968)

IDA*

Iterative-deepening DFS bounded by f-threshold. Memory O(d) - the only practical option at depth 80.

Korf (1985)

Admissibility

h(n) <= true cost from n. Required for optimality. Empirically signals as identical solution depths across heuristics.

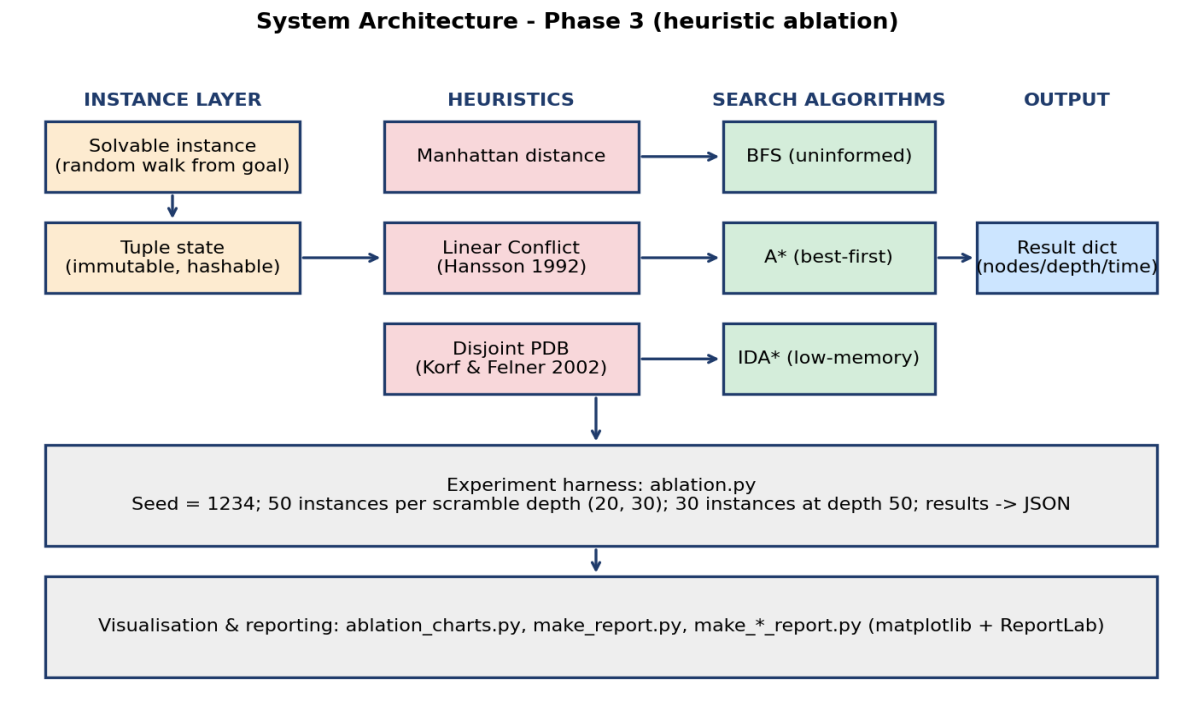
Russell & Norvig (2021)

Heuristic quality

Closer h is to true cost, fewer nodes A* expands. Stronger heuristics trade compute for fewer expansions.

Pearl (1984)

System design: pluggable heuristic architecture



Key idea

- Solvers accept a heuristic callable, not a hard-coded function.
- Adding a new heuristic = one import line, one CLI flag.
- Same code path tested in BFS, A*, and IDA*.
- Ablation becomes a one-line change.

Three admissible heuristics

Manhattan distance

- Sum |dr| + |dc| per tile
- O(n^2) evaluation
- Ignores tile interactions
- Baseline for all comparisons

Russell & Norvig 2021

Linear Conflict

- Manhattan + 2 per row/col conflict
- Greedy max-conflict removal
- Captures pairwise interactions
- Typical 3-5x speedup on 4x4

Hansson, Mayer & Yung 1992

Disjoint PDB (4-4-4-3)

- 4 disjoint tile groups
- 0-1 BFS in projected space
- Sum of group lookups (additive)
- 1.6 M entries, ~4 s build

Korf & Felner 2002

Experimental setup

Protocol

- Random walk from goal -> guaranteed solvable instances
- Three scramble depths: 20, 30, 50
- 50 instances per cell at d=20/30; 30 at d=50
- Per-search caps: 5-10 M node expansions
- Bootstrap mean and 95% CI, 1000 resamples
- Pure Python 3.9 - no C extensions

Reproducibility seeds

Phase 2 main	**42**
3x3 depth sweep	**99**
4x4 depth sweep	**77**
Phase 3 ablation	**1234**

Running any script twice produces identical metrics.

Headline result

15.2x

fewer A* nodes expanded by Disjoint PDB vs Manhattan

at scramble depth 50 - 30 random instances, all solved optimally

9.7x

wall-clock speedup

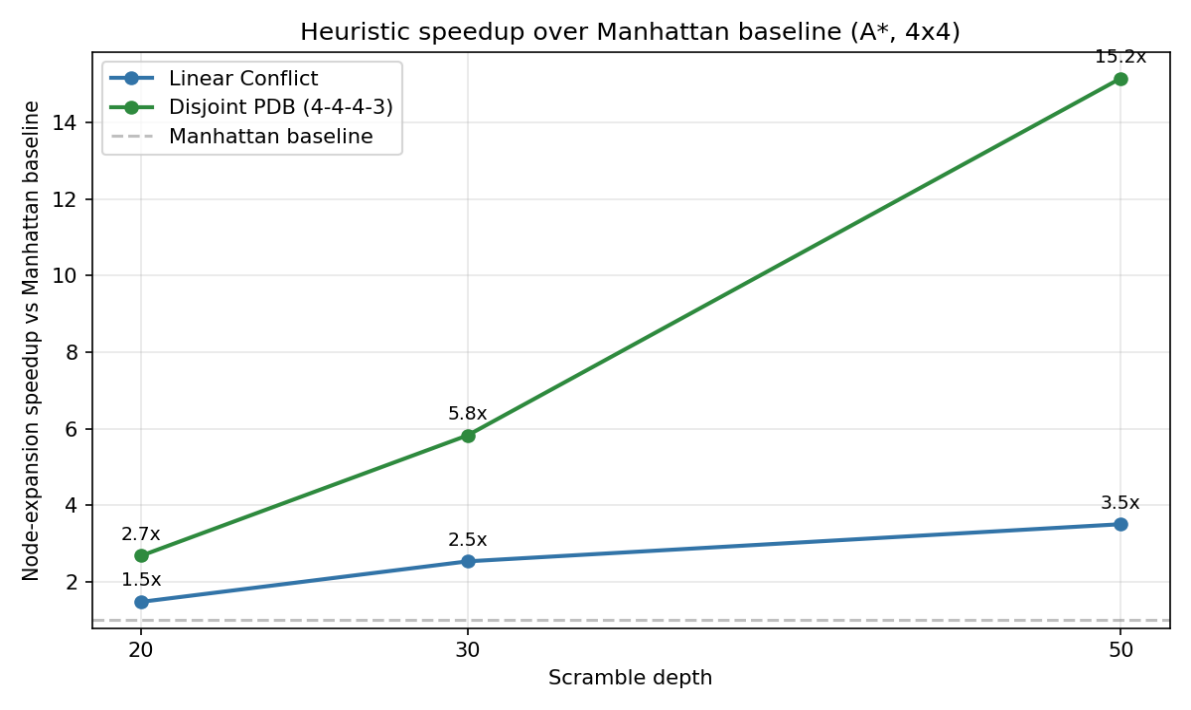
3.5x

Linear Conflict speedup

100%

admissibility preserved

Node-expansion speedup grows with depth



Read

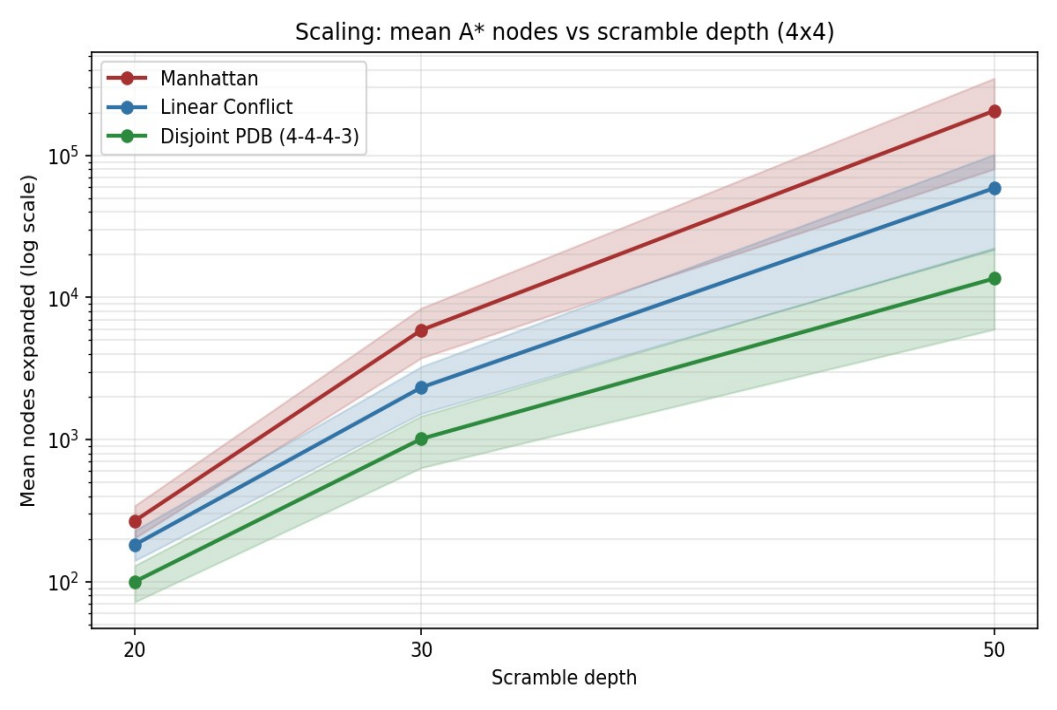
- Linear Conflict: steady 1.5-3.5x
- Disjoint PDB: 2.7x at d=20 grows to 15.2x at d=50
- Gap widens because Manhattan error scales with depth
- PDB captures within-group exact costs

Results: A* nodes expanded with 95% CI

Depth	Heuristic	Solved	Mean nodes	95% CI
20	Manhattan	50/50	268	[203, 343]
20	Linear Conflict	50/50	182	[141, 229]
20	Disjoint PDB	50/50	**100**	**[72, 130]**
30	Manhattan	50/50	5,896	[3,758, 8,425]
30	Linear Conflict	50/50	2,328	[1,528, 3,263]
30	Disjoint PDB	50/50	**1,012**	**[634, 1,467]**
50	Manhattan	30/30	206,058	[80,300, 348,433]
50	Linear Conflict	30/30	58,840	[21,729, 101,422]
50	Disjoint PDB	30/30	**13,600**	**[5,980, 22,300]**

Solution depths matched across all heuristics - admissibility confirmed empirically.

Why does PDB win at depth?



Three reinforcing reasons

Heuristic error.

Manhattan ignores tile interactions; its error grows roughly linearly with depth.

Captured structure.

PDB stores exact within-group cost - the interaction Manhattan misses.

Per-call cost.

PDB lookup = single dict access. Linear Conflict pays an inner loop per call.

What is left on the table

Pure Python overhead

Absolute runtimes lag C/C++; ratios are the meaningful quantity.

Non-canonical partition

4-4-4-3 fits in 1.6 M entries. 7-8 partition would dominate but needs a multi-hour build.

Depth ceiling

Only random instances tested up to d=50. Pathological d~80 instances unexplored.

No tie-breaking

Asai-Fukunaga prefer-larger-g shown to help. Not implemented in this deliverable.

Takeaways

1 **Pluggable heuristics make ablation cheap.**

One callable, one CLI flag - nothing else changes.

2 **Disjoint PDB delivers 15.2x at d=50.**

Wall-clock speedup of 9.7x. Strict optimality preserved.

3 **Pure Python is enough for the science.**

Ratios are reproducible; 4 s build time keeps grading easy.

Future work

Canonical 5-5-5 / 7-8 PDB | Asai-Fukunaga tie-breaking | Transposition tables in IDA* | Generalise to 24-puzzle

Repo: github.com/AnSikwa/Heuristics-Project_Anthony-Kwasi